

SQL (Structured Query Language) is a standard language for retrieving, managing and organizing data stored in databases.

What is a database?

Definition:

Databases are systems that allow users to store and organize data.

They are useful when dealing with large amounts of data.

Databases have wide variety of users!

Analysts (Marketing, Business and Sales)

Technical (Data Engineer, Data Scientist, Data Analyst, Software Engineers, Web Developers). Basically anyone needed to deal with data!

And there are two common types of databases:

1. Non-Relational
2. Relational (where SQL is used)

Database Platforms:

PostgreSQL

MySQL

MariaSQL

MS SQL Server Express

Microsoft Access

SQLite

PostgreSQL is a great database choice for learning how to use SQL.

SQL learned here can be applied to a variety of Databases or SQL based software!

SQL (Structured Query Language) is a standard language for retrieving, managing and organizing data stored in databases.

SQL is the programming language used to communicate with our database.

SQL is used to interact with and manage data stored in **relational databases**. This includes:

- retrieving data from a database table
- inserting data in a database table
- updating data in a database table
- deleting data in a database table
- creating new tables in a database
- and many more

Writing Clean SQL Code

Writing clean SQL code is essential for readability and maintainability. The following tips apply to SQL in general, not just SQL SELECT statements:

- Use Capital Letters for Keywords
 - SQL is not case sensitive. However, it is better to use **capital letters** for keywords such as SELECT and FROM to make the query more readable.
- Use Proper Spacing
 - While it is possible to write SQL queries on a single line, using proper **indentation** and **whitespace** greatly enhances readability.
- Use Comments for Long Queries

Comments are helpful descriptions that explain what the SQL code is doing. They are ignored by the database system during execution. In SQL, any text between -- and the end of the line is treated as a comment.

SQL Example:

```
SELECT customer_id, first_name, last_name  
FROM sales  
ORDER BY first_name;
```

Relational Database:

In a relational database, data is stored in a tabular format.

Name	FName	City	Age	Salary
Smith	John	3	35	\$280
Doe	Jane	1	28	\$325
Brown	Scott	3	41	\$265
Howard	Shemp	4	48	\$359
Taylor	Tom	2	22	\$250

The first row of the table is **fields** or **attributes**.

Installation and Setup of PostgreSQL:

Step 1 : Download and Install PostgreSQL

The screenshot shows a Google search interface with the query 'postgresql'. The search results display the official PostgreSQL website as the top result. The website's title is 'PostgreSQL: The world's most advanced open source database'. Below the title, there are several links and descriptions: 'Downloads' (PostgreSQL is available for download as ready-to-use ...), 'Documentation' (Manuals. You can view the manual for an older version or ...), 'Windows installers' (Interactive installer by EDB. Download the installer certified ...), 'macOS packages Apple Logo' (Postgres.app. Postgres.app is a simple, native macOS app that ...), and 'PostgreSQL 17.2 Documentation' (The PostgreSQL Global Development Group · 1. Getting ...). At the bottom, there is a link to 'More results from postgresql.org »'.



Downloads

PostgreSQL Downloads

PostgreSQL is available for download as ready-to-use packages or installers for various platforms, as well as a source code archive if you want to build it yourself.

Packages and Installers

Select your operating system family:



Source code

The source code can be found in the main [file browser](#) or you can access the source control repository directly at git.postgresql.org. Instructions for building from source can be found in the [documentation](#).

Beta/RC Releases and development snapshots (unstable)

There are source code and binary [packages](#) of beta and release candidates, and of the current development code available for testing and evaluation of new features. Note that these builds should be used **for testing purposes only**, and not for production systems.

3rd party distributions

Windows installers

Interactive installer by EDB

Download the [installer](#) certified by EDB for all supported PostgreSQL versions.

Note! This installer is hosted by EDB and not on the PostgreSQL community servers. If you have issues with the website it's hosted on, please contact webmaster@enterprisedb.com.

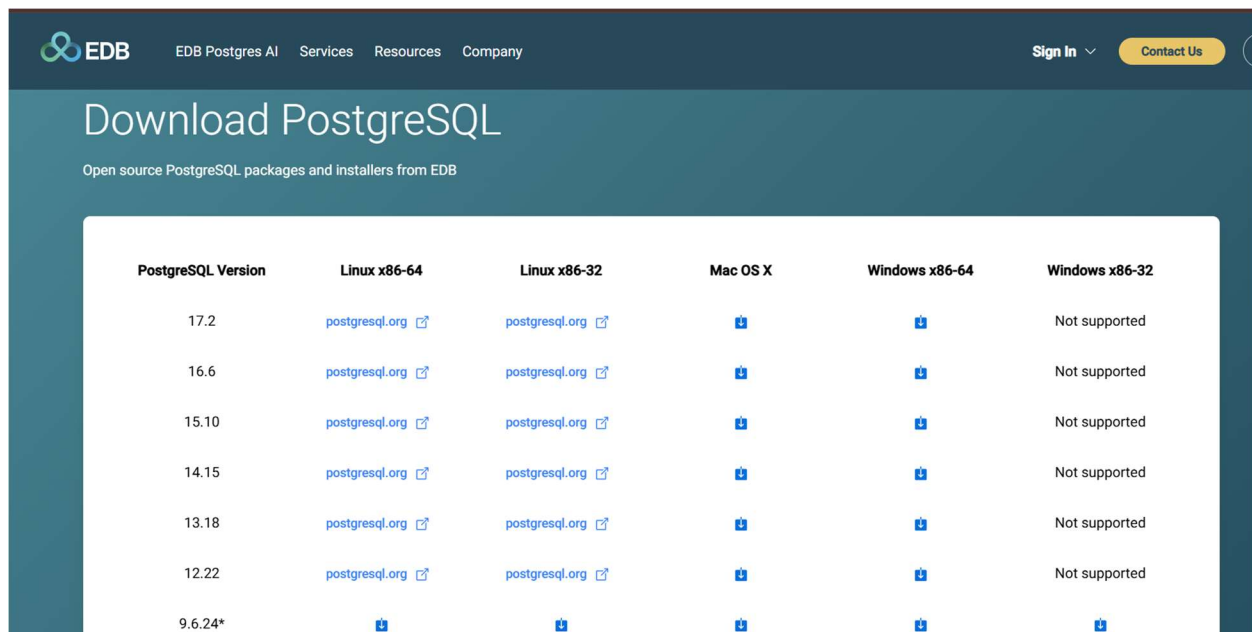
This installer includes:

- The PostgreSQL server
- pgAdmin, a graphical tool for managing and developing your databases
- StackBuilder, a package manager for downloading and installing additional PostgreSQL tools and drivers. Stackbuilder includes management, integration, migration, replication, geospatial, connectors and other tools.

This installer can run in graphical or silent install modes.

The Windows installer is designed to be a straightforward, fast way to get up and running with PostgreSQL. In the unlikely event that you have issues with the installer, please report them on the [installer Github page](#). Issues with the installed packages should be reported to the appropriate project directly.

Advanced users can also download a [zip archive](#) of the binaries, without the installer. This download is intended for users who wish to include PostgreSQL as part of another application installer.



After downloading it install the application:

Repeat Next, Next ...

Put the password

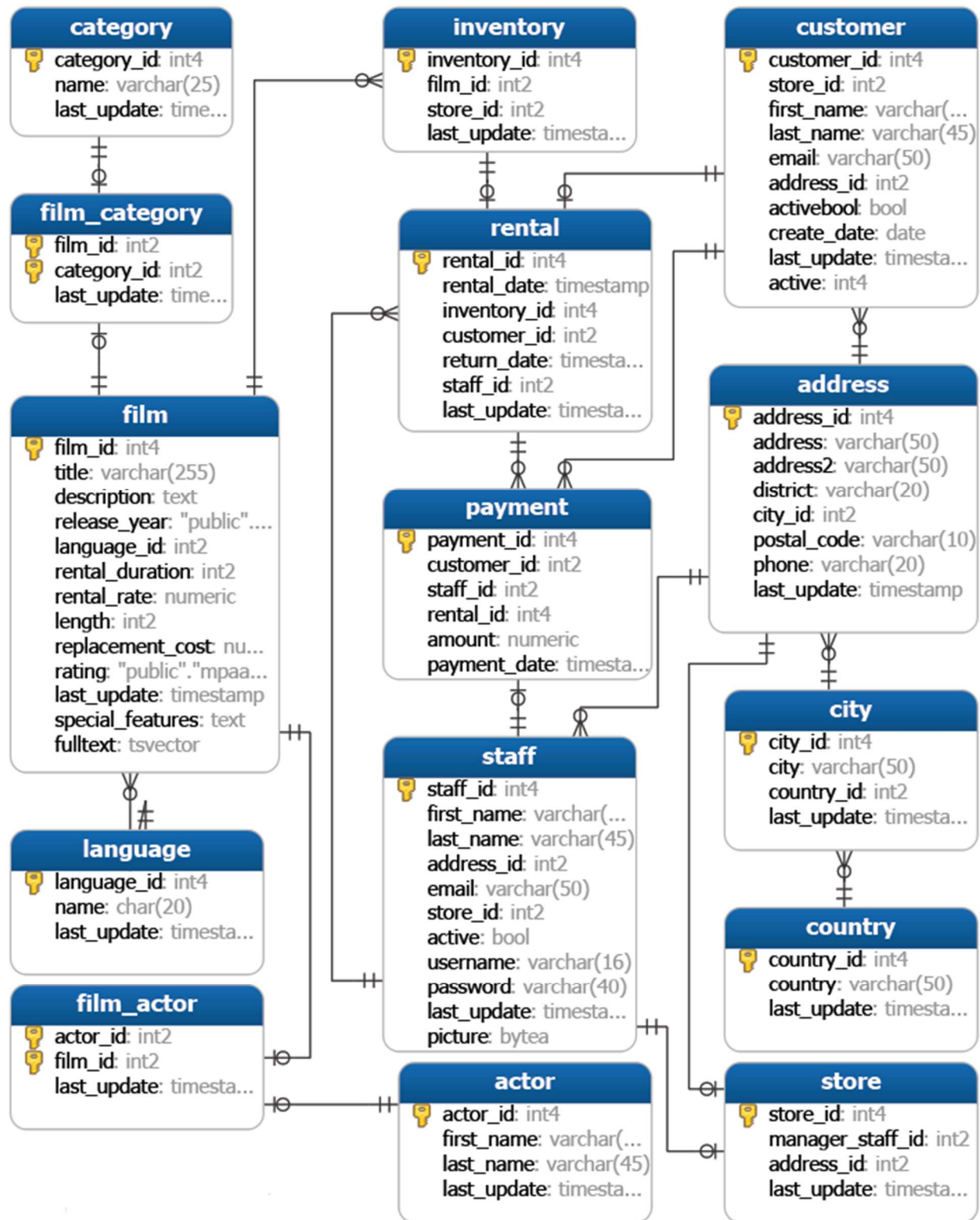
After finishing the installing untick the button before Stack Builder

Finish

I will send dvdrental.tar file but don't directly open it.

This file is the compressed version of the database that we will use further.

Restart your laptop.



Database Schema

Go to search box and search pgAdmin and click it. This will take some time.

After this double click on your PostgreSQL version and enter the password.

Right Click → Create → Database

Name the database as dvdrental and save.

Right click on dvdrental database and click on restore.

Put the location of file on filename and go on restore option.

Make pre-data , data and post-data yes no sections and finally click restore.

Right click on devrental and refresh.

In order to run the SQL query right click on dvdrental and click on Query Tool.

SELECT *

FROM film;

SQL SELECT STATEMENTS	SQL UPDATE TABLE	DROP TABLE tbl
SELECT * FROM tbl Select all rows and columns from table tbl	INSERT INTO tbl(c1,c2,...) VALUES(v1,v2,...) Insert data into table tbl	Remove table tbl from database.
SELECT c1,c2 FROM tbl Select column c1, c2 and all rows from table tbl	INSERT INTO tbl(c1,c2,...) SELECT c1,c2.. FROM tbl2 WHERE conditions Insert data from tbl2 into tbl	ALTER TABLE tbl ADD COLUMN c1 datatype(length) Add column c1 to table tbl
SELECT c1,c2 FROM tbl WHERE conditions ORDER BY c1 ASC, c2 DESC Select columns c1, c2 with where conditions and from table tbl order result by column c1 in ascending order and c2 in descending order	UPDATE t SET c1 = v1, c2 = v2... WHERE conditions Update data in table tbl	ALTER TABLE tbl DROP COLUMN c1 Drop column c1 from table tbl
SELECT DISTINCT c1, c2 FROM tbl Select distinct rows by columns c1 and c2 from table tbl.	DELETE FROM tbl WHERE conditions Delete records from table tbl based on WHERE conditions.	SQL JOIN STATEMENTS
SELECT c1, aggregate(expr) FROM tbl GROUP BY c1 Select column c1 and use aggregate function on expression expr; group columns by column c1.	TRUNCATE TABLE tbl Drop table tbl and re-create it, all data is lost	SELECT * FROM tbl1 INNER JOIN tbl2 ON join-conditions Inner join table tbl1 with tbl2 based on join-conditions.
SELECT c1, aggregate(expr) AS c2 FROM tbl GROUP BY c1 HAVING c2 > v Select column c1 and c2 as column alias of the result of aggregate function on expr. Filter group of records with c2 greater than value v	SQL TABLE STATEMENTS	SELECT * FROM tbl1 LEFT JOIN tbl2 ON join-conditions Left join table tbl1 with tbl2 based on join-conditions.
	CREATE TABLE tbl(c1 datatype(length) c2 datatype(length) ... PRIMARY KEY(c1)) Create table tbl with primary key is c1	SELECT * FROM tbl1 RIGHT JOIN tbl2 ON join-conditions Right join table tbl1 with tbl2 based on join-conditions.
		SELECT * FROM tbl1 RIGHT JOIN tbl2 ON join-conditions Full outer join table tbl1 with tbl2 based on join-conditions.

Challenges:

The basis of the challenge scenarios will be that you have been hired as a SQL consultant for a DVD rental store.

SQL Statement Fundamentals:

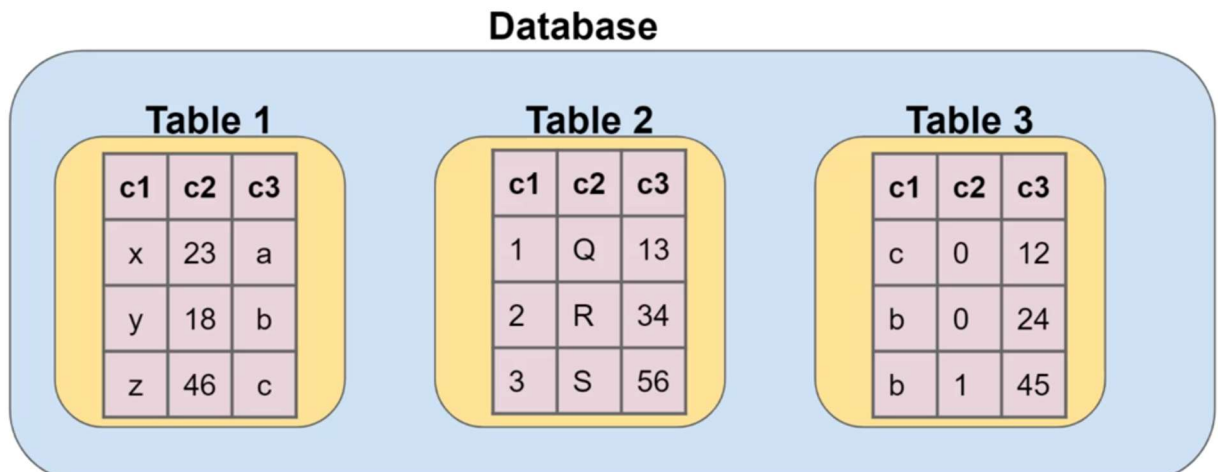
1) SELECT Statement

SELECT is the most common statement used and it allows us to retrieve information from a table.

Syntax for SELECT statement:

SELECT column_name

FROM table_name



```
SELECT c1
```

```
FROM Table1
```

```
SELECT c1,c2
```

```
FROM Table1
```

```
SELECT c1,c3
```

```
FROM Table1
```

```
SELECT *
```

```
FROM Table1
```

In general it is not good practice to use an asterisk(*) in the SELECT statement if you don't really need all columns.

If you want to know the tables stored in the database click on Schemas and then Tables.

Queries :

a) Select all the columns from the table

b) SELECT *

FROM actor;

- c) Select a specific single column from the table

```
SELECT first_name  
FROM actor;
```

- d) Select specific columns from the table

```
SELECT first_name,last_name  
FROM actor;
```

Challenge

Q) We want to send out a promotional email to our existing customers!

Hint: For solving this challenge we need to know name and email of the customers

Use a SELECT statement to grab the first and last names of every customer and their email address.

Use the customer table.

You can use the table drop-down to view what columns are available.

You could also use SELECT * FROM customer to see all the columns.

Output:

Data Output Messages Notifications			
Showing rows: 1 to 599			
	first_name character varying (45)	last_name character varying (45)	email character varying (50)
1	Jared	Ely	jared.ely@sakilacustomer.org
2	Mary	Smith	mary.smith@sakilacustomer.org
3	Patricia	Johnson	patricia.johnson@sakilacustomer.org
4	Linda	Williams	linda.williams@sakilacustomer.org
5	Barbara	Jones	barbara.jones@sakilacustomer.org
6	Elizabeth	Brown	elizabeth.brown@sakilacustomer.org
7	Jennifer	Davis	jennifer.davis@sakilacustomer.org
8	Maria	Miller	maria.miller@sakilacustomer.org
9	Susan	Wilson	susan.wilson@sakilacustomer.org
10	Margaret	Moore	margaret.moore@sakilacustomer.org
Total rows: 599		Query complete 00:00:00.218	

2) SELECT DISTINCT Statement

Sometimes a table contains a column that has duplicate values, and you may find yourself in a situation where you only want to list the unique/distinct values.

The DISTINCT keyword can be used to return only the distinct values in a column.

The DISTINCT keyword operates on a column.

Syntax:

```
SELECT DISTINCT column
```

```
FROM table;
```

Or, you can also use small bracket as

```
SELECT DISTINCT (column)
```

```
FROM table;
```

SELECT DISTINCT name FROM color_table

Name	Choice
Zach	Green
David	Green
Claire	Yellow
David	Red

SELECT DISTINCT choice FROM color_table

Name
Zach
David
Claire

Choice
Green
Yellow
Red

Select all the columns of film table.

```
SELECT *
```

```
FROM film;
```

Select unique release year from the film table.

```
SELECT DISTINCT release_year
```

```
FROM film;
```

Select unique rental rate from the film table.

```
SELECT DISTINCT rental_rate  
  
FROM film;
```

Challenges

Q) Use what we have learned about DISTINCT to retrieve the different rating types our films could have in our database.

Hint:

Use the film table.

You can use the table drop-down to view what columns are available.

You could also use SELECT * FROM films to see all the columns.

Output:

	rating mpaa_rating 
1	NC-17
2	G
3	PG
4	PG-13
5	R

3) COUNT Function

The COUNT function returns the number of input rows that match a specific condition of a query.

We can apply COUNT on a specific column or just pass COUNT (*).

SELECT COUNT(name) FROM table;

Name	Choice
Zach	Green
David	Green
Claire	Yellow
David	Red

Count
4

Syntax:

```
SELECT COUNT(Name)
FROM table; Or,
SELECT COUNT(Choice)
FROM table; Or,
SELECT COUNT(*)
FROM table;
This is simply returning the number of rows in the table.
```

How many unique names are there in the table?

```
SELECT COUNT(DISTINCT Name)
FROM table;
```

Count
3

Select all columns from payment table.

```
SELECT *
FROM payment;
```

```
SELECT COUNT (*)
FROM payment;
```

```
SELECT COUNT (amount)
FROM payment;
```

```
SELECT amount
FROM payment;
```

```
SELECT DISTINCT (amount)
FROM payment;
```

```
SELECT COUNT (DISTINCT amount)
FROM payment;
```

4) SELECT WHERE statement

SELECT and WHERE the most fundamental SQL statements.

The WHERE statement allows us to specify conditions on columns for the rows to be returned.

The WHERE clause appears immediately after the FROM clause of the SELECT statement.

The conditions are used to filter the rows returned from the SELECT statement.

Syntax:

```
SELECT column1, column2
```

```
FROM table
```

```
WHERE conditions;
```

Comparison operators compare a column value to something.

For ex: Is the price greater than Rs.100?

Comparison Operators

Operator	Description
=	Equal
>	Greater than
<	Less Than
>=	Greater than or equal to
<=	Less than or equal to
<> or !=	Not equal to

Logical Operators

- Allow us to combine multiple comparison operators
 - **AND**
 - **OR**
 - **NOT**

```
SELECT Name, Choice  
FROM table;
```

Name	Choice
Zach	Green
David	Green
Claire	Yellow
David	Red

```
SELECT Name, Choice  
FROM table  
WHERE name = 'David';
```

Name	Choice
David	Green
David	Red

```
SELECT Name  
FROM table  
WHERE name = 'David';
```

Name
David
David

```
SELECT Name, Choice
FROM table
WHERE name = 'David' AND choice = 'Red';
```

Name	Choice
David	Red

```
SELECT *
FROM customer;
```

```
SELECT *
FROM customer
WHERE first_name = 'Jared';
```

```
SELECT *
FROM film;
```

```
SELECT *
FROM film
WHERE rental_rate > 4;
```

```
SELECT *
FROM film
WHERE rental_rate > 4
AND replacement_cost >= 19.99;
```

```
SELECT *
FROM film
WHERE rental_rate > 4
AND replacement_cost >= 19.99
AND rating = 'R';
```

```
SELECT title
FROM film
WHERE rental_rate > 4
```



```
AND replacement_cost >= 19.99  
AND rating = 'R';
```

```
SELECT COUNT (title)  
FROM film  
WHERE rental_rate > 4  
AND replacement_cost >= 19.99  
AND rating = 'R';
```

```
SELECT COUNT (*)
FROM film
WHERE rental_rate > 4
AND replacement_cost >= 19.99
AND rating = 'R';
```

```
SELECT COUNT (*)
FROM film
WHERE rating = 'R'
OR rating = 'PG-13';
```

Challenge

Q) What is the email for the customer with the name Nancy Thomas?

Hints:

Use the customer table.

Make sure the capitalization and spelling of the names is correct.

Use AND to combine conditions.

Use single quotes around the 'string'.

Output:

Showing rows: 1 to 1		Page No: 1
	email character varying (50)	
1	nancy.thomas@sakilacustomer.org	

Q) Could you give the description for the movie "Outlaw Hanky"?

Hints:

Use the film table.

Make sure the capitalization and spelling of the movie name is correct.

Use single quotes around the 'string'.

Output:

	description text	
1	A Thoughtful Story of a Astronaut And a Composer who must Conquer a Dog in The Sahara Des...	

Q) Can you get the phone number for the customer who lives at '259 Ipoh Drive'?

Hints:

Use the address table.

Make sure the capitalization and spelling of the address is correct.

Use single quotes around the 'string'.

Output:

	phone character varying (20) 
1	419009857119

5) ORDER BY

PostgreSQL sometimes returns the same request query results in a different order.

We can use ORDER BY to sort rows based on a column value, in either ascending or descending order.

Order will be alphabetical for string based columns and numerical order for numeric columns.

Order by will be towards the bottom of a query and one of the very last things that SQL actually performs.

Syntax:

```
SELECT column_1, column_2
```

```
FROM table
```

```
ORDER BY column_1 ASC/DESC
```

Use ASC to sort in ascending order.

Use DESC to sort in descending order.

If we leave it blank, ORDER BY uses ASC by default.

We can also ORDER BY multiple columns.

This makes sense when one column has duplicate entries.

Company	Name	Sales
Apple	Andrew	100
Google	David	500
Apple	Zach	300
Google	Claire	200
Xerox	Steven	100

```
SELECT Company, Name, Sales
FROM table ORDER BY Company, Sales
```

Company	Name	Sales
Apple	Andrew	100
Apple	Zach	300
Google	Claire	200
Google	David	500
Xerox	Steven	100

```
SELECT *
FROM customer;
```

```
SELECT *
FROM customer
ORDER BY first_name;
```

```
SELECT *
FROM customer
ORDER BY first_name ASC;
```

```
SELECT *
FROM customer
ORDER BY first_name DESC;
```

```
SELECT *
FROM customer
ORDER BY store_id;
```

```
SELECT store_id, first_name, last_name
FROM customer
ORDER BY store_id;
```

```
SELECT store_id, first_name, last_name
FROM customer
ORDER BY store_id, first_name;
```

```
SELECT store_id, first_name, last_name
FROM customer
ORDER BY store_id DESC, first_name ASC;
```

```
SELECT first_name, last_name
FROM customer
ORDER BY store_id DESC, first_name ASC;
```

6) LIMIT

LIMIT command allows us to limit the number of rows returned for a query.

It is very useful if we don't want to return every single row in a table, but only view the top few rows to get an idea of the table layout.

LIMIT also becomes useful in combination with ORDER BY.

LIMIT goes at the very end of the query request and is the last command to be executed.

```
SELECT *
FROM payment;
```

```
SELECT *
FROM payment
ORDER BY payment_date;
```

```
SELECT *
FROM payment
ORDER BY payment_date ASC;
```

```
SELECT *
FROM payment
ORDER BY payment_date DESC;
```

```
SELECT *
FROM payment
ORDER BY payment_date DESC
LIMIT 5;
```

```
SELECT *
FROM payment
WHERE amount! = 0.00
ORDER BY payment_date DESC
LIMIT 5;
```

Challenge:

What are the customer ids of the first 10 customers who created a payment?

Hints:

Use the payment table.

We need to use both ORDER BY and LIMIT.

Remember we may need to specify ASC or DESC.

Output:

		     	
	customer_id	smallint 	
1		416	
2		516	
3		239	
4		592	
5		49	
6		264	
7		46	
8		481	
9		139	
10		595	
Total rows: 10		Query c	

Q) What are the titles and length of the 5 shortest (in length of runtime) movies?

Hints:

Use the film table.

Take a look at the length column.

We can use ORDER BY and LIMIT.

Remember to use ASC or DESC to get desired results.

Output:

	title character varying (255) 🔒	length smallint 🔒
1	Labyrinth League	46
2	Alien Center	46
3	Iron Moon	46
4	Kwai Homeward	46
5	Ridgemont Submarine	46

Q) How many movies can the customer watch that is 50 minutes or less in runtime?

	count bigint 🔒
1	37

7) BETWEEN operator

The BETWEEN operator can be used to match a value against a range of values:

value BETWEEN low AND high. BETWEEN is inclusive and NOT BETWEEN is exclusive.

The BETWEEN operator is same as:

- value $\geq$ low AND value $\leq$ high
- value BETWEEN low AND high

We can also combine BETWEEN with the NOT logical operator:

- value NOT BETWEEN low AND high

The NOT BETWEEN operator is the same as:

- value $<$ low OR value $>$ high
- value NOT BETWEEN low AND high

The BETWEEN operator can also be used with dates.

We need to format dates into the ISO 8601 standard format, which is YYYY-MM-DD

- date BETWEEN '2007-01-01' AND '2007-02-01'.

SELECT *

FROM payment

LIMIT 2;

SELECT *

FROM payment

WHERE amount

BETWEEN 8 AND 9;

SELECT COUNT(*)

FROM payment

WHERE amount

BETWEEN 8 AND 9;

SELECT COUNT(*)

FROM payment

WHERE amount

NOT BETWEEN 8 AND 9;

SELECT *

FROM payment

WHERE payment_date

BETWEEN '2007-02-01' AND '2007-02-15';

SELECT *

FROM payment

WHERE payment_date

BETWEEN '2007-02-01' AND '2007-02-14';

8) IN

We can use the IN operator to create a condition that checks to see if a value is included in a list of multiple options.

Syntax: value IN(option1,option2,.....,option_n)

Example Query :

```
SELECT color FROM table  
WHERE color IN('red','blue');
```

```
SELECT color FROM table  
WHERE color IN('red','blue','green');
```

```
SELECT color FROM table  
WHERE color NOT IN('red','blue');
```

```
SELECT *  
FROM payment  
LIMIT 2;
```

```
SELECT DISTINCT(amount)  
FROM payment;
```

```
SELECT DISTINCT(amount)  
FROM payment  
ORDER BY amount;
```

```
SELECT *  
FROM payment  
WHERE amount IN(0.99,1.98,1.99);
```

```
SELECT COUNT(*)  
FROM payment  
WHERE amount IN(0.99,1.98,1.99);
```

```
SELECT COUNT(*)  
FROM payment  
WHERE amount NOT IN(0.99,1.98,1.99);
```

```
SELECT *  
FROM customer  
WHERE first_name IN ('John','Jake','Julie');
```

```
SELECT *  
FROM customer  
WHERE first_name NOT IN ('John','Jake','Julie');
```

9) LIKE and ILIKE

We have already been able to perform direct comparisons against strings, such as :

- WHERE first_name = 'John'

But what if we want to match against a general pattern in a string?

- All emails ending in '@gmail.com'.
- All names that begin with an 'A'.

The LIKE operator allows us to perform pattern matching against string data with the use of wildcard characters :

- Percent % (matches any sequences of characters)
- Underscore _ (matches any single character)

For ex :

- All names that begin with 'A' → WHERE name LIKE 'A%'.
- All names that end with 'a' → WHERE name LIKE '%a'.

Notice that LIKE is case-sensitive, we can use ILIKE which is case-insensitive.

Using the underscore allows us to replace just a single character.

Get all Mission Impossible films.

- WHERE title LIKE 'Mission Impossible _'.

We can use multiple underscores.

Imagine we had version string codes in the format 'Version#A4','Version#B7',etc ...

- WHERE value LIKE 'Version#_ _'.

We can also combine pattern matching operators to create more complex patterns.

- WHERE name LIKE '_her%'
 - o Cheryl
 - o Sherri
 - o Theresa

SELECT *

FROM customer

WHERE first_name LIKE 'J%';

SELECT COUNT(*)

FROM customer

WHERE first_name LIKE 'J%';

```
SELECT *  
FROM customer  
WHERE first_name LIKE 'J%' AND last_name LIKE 'S%';
```

```
SELECT *  
FROM customer  
WHERE first_name LIKE 'j%' AND last_name LIKE 'j%';
```

```
SELECT *  
FROM customer  
WHERE first_name LIKE '%er%';
```

```
SELECT *  
FROM customer  
WHERE first_name LIKE 'A%' AND last_name NOT LIKE 'B%'  
ORDER BY last_name;
```

General Challenges)

Q) How many payment transactions were greater than \$5.00?

Hints:

We need to use the payment table.

We also need to use COUNT and WHERE along with some comparison operator.

Q) How many actors have a first name that starts with the letter P?

Hints:

We need to use the actor table.

We also need to use LIKE and a wildcard operator such as % or _.

Q) How many unique districts are our customers from?

Hints:

We need to use the address table.

We also need to use COUNT and DISTINCT.

Q) How many films have a rating of R and a replacement cost between \$5 and \$15?

Hints:

We need to use the film table.

We also need to use BETWEEN and a WHERE statement with a comparison operator.

Q) How many films have the word Truman somewhere in the title?

Hints:

We need to use the film table.

We also need to use LIKE with a wildcard operator.

Aggregation Functions

SQL provides a variety of aggregate functions.

The main idea behind an aggregate function is to take multiple inputs and return a single output.

Aggregate function calls happen only in the SELECT clause or the HAVING clause.

Most Common Aggregate Functions are:

- AVG() → returns average value
- COUNT() → returns number of values
- MAX() → returns maximum value
- MIN() → returns minimum value
- SUM() → returns the sum of all values

Special Notes

AVG () returns a floating point value many decimal places (2.335455)

We can use ROUND () to specify precision after the decimal.

COUNT () simply returns the number of rows, which means by convention we just use COUNT (*).

SELECT *

FROM film;

```
SELECT MIN(replacement_cost)
FROM film;
```

```
SELECT MAX(replacement_cost)
FROM film;
```

```
SELECT MAX(replacement_cost), MIN(replacement_cost)
FROM film;
```

```
SELECT COUNT(*)
FROM film;
```

```
SELECT AVG(replacement_cost)
FROM film;
```

```
SELECT ROUND(AVG(replacement_cost),2)
FROM film;
```

```
SELECT ROUND(AVG(replacement_cost),4)
FROM film;
```

```
SELECT SUM(replacement_cost)
FROM film;
```

JOINS

JOINS will allow us to combine information from multiple tables.

There are different types of joins:

- INNER JOINS
- OUTER JOINS
- FULL JOINS
- UNIONS

AS Statement

AS clause allows us to create an “alias” for a column or result.

Syntax:

```
SELECT column
```

```
AS new_name
```

```
FROM table;
```

```
SELECT SUM(column)
```

```
AS new_name
```

```
FROM table;
```

```
SELECT customer_id
```

```
FROM payment;
```

```
SELECT amount
```

```
FROM payment;
```

```
SELECT amount
```

```
AS rental_price
```

```
FROM payment;
```

```
SELECT SUM(amount)
```

```
AS net_revenue
```

FROM payment;

JOINS allow us to combine multiple tables together.

The main reason for the different JOIN types is to decide how to deal with information only present in one of the joined tables.

Let's imagine a simple example.

Our company is holding a conference for people in the movie rental industry.

We will have people register online beforehand and the login the day of the conference.

After the conference we have these tables.

REGISTRATIONS		LOGINS	
reg_id	name	log_id	name
1	Andrew	1	Xavier
2	Bob	2	Andrew
3	Charlie	3	Yolanda
4	David	4	Bob

The respective id columns indicate what order they registered or logged in on site.

a) INNER JOIN

An INNER JOIN will result with the set of records that match in both tables.

If we see just JOIN without the INNER, PostgreSQL will treat it as an INNER JOIN.

REGISTRATIONS		LOGINS	
reg_id	name	log_id	name
1	Andrew	1	Xavier
2	Bob	2	Andrew
3	Charlie	3	Yolanda
4	David	4	Bob

```
SELECT * FROM TableA
```

```
INNER JOIN TableB
```

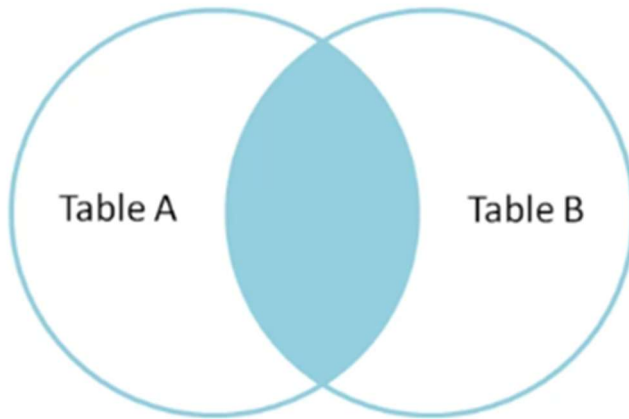
```
ON TableA.col_match = TableB.col_match;
```

This is similar to :

```
SELECT * FROM TableB
```

```
INNER JOIN TableA
```

```
ON TableA.col_match = TableB.col_match;
```

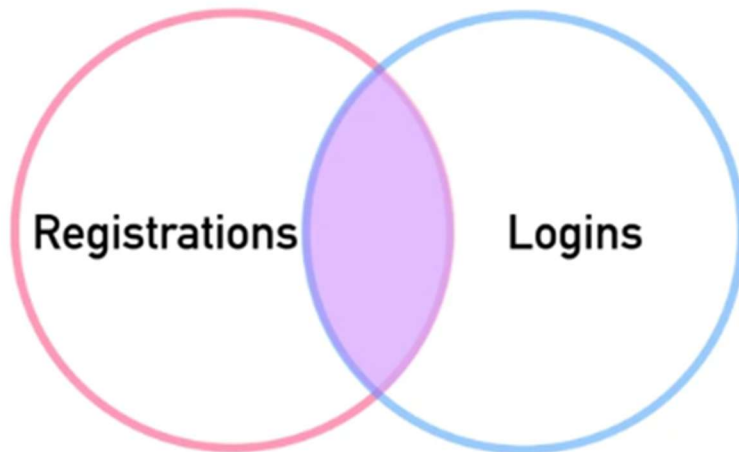


REGISTRATIONS		LOGINS	
reg_id	name	log_id	name
1	Andrew	1	Xavier
2	Bob	2	Andrew
3	Charlie	3	Yolanda
4	David	4	Bob

```
SELECT * FROM REGISTRATIONS
```

```
INNER JOIN LOGINS
```

```
ON REGISTRATIONS.name = LOGINS.name;
```

RESULTS			
reg_id	name	log_id	name
1	Andrew	2	Andrew
2	Bob	4	Bob

```
SELECT reg_id, LOGINS.name, log_id
FROM REGISTRATIONS
INNER JOIN LOGINS
ON REGISTRATIONS.name = LOGINS.name;
```

RESULTS		
reg_id	name	log_id
1	Andrew	2
2	Bob	4

Let's practice on PostgreSQL:

```
SELECT * FROM payment
```

```
INNER JOIN customer
ON payment.customer_id = customer.customer_id;

SELECT payment_id, payment.customer_id, first_name
FROM payment
INNER JOIN customer
ON payment.customer_id = customer.customer_id;
```

CREATING DATABASES AND TABLES:

DATA TYPES

- Boolean - True or False
- Character - char, varchar and text
- Numeric – integer and floating point number
- Temporal – date, time, timestamp and interval

When creating databases and tables, we should carefully consider which data types should be used for the data to be stored.

Primary Key :

Primary key is a column or a group of columns used to identify a row uniquely in a table.

It must be unique and not null.

For ex : in our dvdrental database we saw customers had a unique, non-null customer_id column as their primary key.

```
SELECT *
FROM customer;
```

	customer_id [PK] integer	store_id smallint	first_name character varying (45)	last_name character varying (45)	email character varying (50)	address_id smallint	activebool boolean	create_date date	last_upd timestar
4	3	1	Linda	Williams	linda.williams@sakilacustomer.org	7	true	2006-02-14	2013-05
5	4	2	Barbara	Jones	barbara.jones@sakilacustomer.org	8	true	2006-02-14	2013-05
6	5	1	Elizabeth	Brown	elizabeth.brown@sakilacustomer.org	9	true	2006-02-14	2013-05
7	6	2	Jennifer	Davis	jennifer.davis@sakilacustomer.org	10	true	2006-02-14	2013-05
8	7	1	Maria	Miller	maria.miller@sakilacustomer.org	11	true	2006-02-14	2013-05
9	8	2	Susan	Wilson	susan.wilson@sakilacustomer.org	12	true	2006-02-14	2013-05
10	9	2	Margaret	Moore	margaret.moore@sakilacustomer.org	13	true	2006-02-14	2013-05
11	10	1	Dorothy	Taylor	dorothy.taylor@sakilacustomer.org	14	true	2006-02-14	2013-05
12	11	2	Lisa	Anderson	lisa.anderson@sakilacustomer.org	15	true	2006-02-14	2013-05

A foreign key is a field or group of fields in a table that uniquely identifies a row in another table.

A foreign key is defined in a table that references to the primary key of the other table.

Recall in the dvdrental database payment table, each payment row had its unique payment_id(a primary key) and identified the customer that made the payment through the customer_id(a foreign key since it references the customer table's primary key).

customer_id smallint	staff_id smallint	rental_id integer
341	2	1520
341	1	1778
341	1	1849
341	2	2829
341	2	3130
341	1	3382
342	2	2190

customer_id is a foreign key within payment because it is a primary key of the customer table.

staff_id is a foreign key within payment because it is a primary key of staff table.

rental_id is a foreign key within payment because it is a primary key of rental table.

Constraints

Constraints are the rules enforced on data columns on table.

These are used to prevent invalid data from being entered into the database.

This ensures the accuracy and reliability of the data in the database.

There are two types of constraints. They are:

Column Constraints and Table Constraints

The most common constraints are:

NOT NULL Constraint – Ensures that a column cannot have NULL value.

UNIQUE Constraint – Ensures that all values in a column are different.

The most common constraints used:

PRIMARY Key – Uniquely identifies each row in a database table.

FOREIGN Key – Constraints data based on columns in other tables.

CREATE TABLE

Full General Syntax:

```
CREATE TABLE table_name(  
column_name TYPE column_constraint,  
column_name TYPE column_constraint,  
table_constraint table_constraint  
) INHERITS existing_table_name;
```

Common Simple Syntax:

```
CREATE TABLE table_name(  
column_name TYPE column_constraint,  
column_name TYPE column_constraint,  
);
```

```
CREATE TABLE players(  
player_id SERIAL PRIMARY KEY,  
age SMALLINT NOT NULL
```

);

```
CREATE TABLE account(  
    user_id SERIAL PRIMARY KEY,  
    username VARCHAR(50) UNIQUE NOT NULL,  
    password VARCHAR(50) NOT NULL,  
    email VARCHAR(250) UNIQUE NOT NULL,  
    created_on TIMESTAMP NOT NULL,  
    last_login TIMESTAMP  
);
```

```
CREATE TABLE job(  
    job_id SERIAL PRIMARY KEY,  
    job_name VARCHAR(200) UNIQUE NOT NULL  
);
```

```
CREATE TABLE account_job(  
    user_id INTEGER REFERENCES account(user_id),  
    job_id INTEGER REFERENCES job(job_id),  
    hire_date TIMESTAMP  
);
```

INSERT DATA IN THE TABLE

INSERT allows us to add rows to a table.

General Syntax:

```
INSERT INTO table(column1, column2, ...)
```

VALUES

(value1, value2, ...),

(value1, value2, ...) ,...;

Syntax for Inserting Values from another table :

INSERT INTO table(column1, column2, ...)

SELECT column1, column2, ...

FROM another_table

WHERE condition;

SELECT *

FROM account;

INSERT INTO account(username,password,email,created_on)

VALUES

('John','password','john@gmail.com',CURRENT_TIMESTAMP)

SELECT *

FROM account;

SELECT *

FROM job;

INSERT INTO job(job_name)

VALUES

('Data Analyst')

SELECT *

FROM job;

```
INSERT INTO job(job_name)
```

```
VALUES
```

```
('Data Scientist')
```

```
INSERT INTO account_job(user_id, job_id, hire_date)
```

```
VALUES
```

```
(1,1,CURRENT_TIMESTAMP);
```

```
SELECT *
```

```
FROM account_job;
```

```
INSERT INTO account_job(user_id,job_id,hire_date)
```

```
VALUES
```

```
(10,10,CURRENT_TIMESTAMP)
```

```
UPDATE
```

UPDATE keyword allows for the changing of the columns in a table.

General Syntax:

```
UPDATE table
```

```
SET column1 = value1,
```

```
column2=value2,...
```

```
WHERE
```

```
condition;
```

```
UPDATE account
```

```
SET last_login = CURRENT_TIMESTAMP
```

```
WHERE last_login IS NULL;
```

Reset everything without WHERE condition

```
UPDATE account
```

```
SET last_login = CURRENT_TIMESTAMP
```

Set based on another column

```
UPDATE account
```

```
SET last_login = created_on
```

Using another table's values:

```
UPDATE TableA
```

```
SET original_col = TableB.new_col
```

```
FROM tableB
```

```
WHERE tableA.id = TableB.id
```

```
SELECT * FROM account;
```

```
UPDATE account
```

```
SET last_login = CURRENT_TIMESTAMP;
```

```
SELECT * FROM job;
```

```
SELECT * FROM account_job;
```

```
UPDATE account_job
```

```
SET hire_date = account.created_on
```

```
FROM account
```

```
WHERE account_job.user_id = account.user_id;
```

```
UPDATE account
```



```
SET last_login = CURRENT_TIMESTAMP  
RETURNING email,created_on,last_login;
```

DELETE

We can use the DELETE clause to remove rows from a table.

For example:

```
DELETE FROM table  
WHERE row_id = 1;
```

We can delete rows based on their presence in other tables

For example:

```
DELETE FROM tableA  
USING tableB  
WHERE tableA.id=tableB.id;
```

We can delete all rows from a table.

For example:

```
DELETE FROM table
```

Similar to UPDATE command, we can also add in a RETURNING call to return rows that were removed.

```
SELECT *  
FROM job;
```

```
INSERT INTO job(job_name)  
VALUES  
    ('Designer')
```

```
DELETE FROM job  
WHERE job_name = 'Designer'
```

RETURNING job_id, job_name;

ALTER

The ALTER clause allows for changes to an existing table structure, such as :

Adding, dropping or renaming columns

Changing a column's data type

Set DEFAULT values for a column

Add CHECK constraints

Rename table

General Syntax:

ALTER TABLE table_name action

Adding Columns

ALTER TABLE table_name

ADD COLUMN new_col TYPE

Removing Columns

ALTER TABLE table_name

DROP COLUMN col_name

Alter constraints

ALTER TABLE table_name

ALTER COLUMN col_name

SET DEFAULT value

DELETE FROM job

WHERE job_name = 'Designer'

RETURNING job_id, job_name;

```
CREATE TABLE information(  
    info_id SERIAL PRIMARY KEY,  
    title VARCHAR(500) NOT NULL,  
    person VARCHAR(50) NOT NULL UNIQUE  
)
```

```
ALTER TABLE information  
RENAME TO new_info
```

```
SELECT *  
FROM new_info;
```

```
ALTER TABLE new_info  
RENAME COLUMN person TO people;
```

```
SELECT *  
FROM new_info;
```

```
ALTER TABLE new_info  
ALTER COLUMN people DROP NOT NULL
```

DROP

DROP allows for the complete removal of a column in a table.

General Syntax

```
ALTER TABLE table_name
```

```
DROP COLUMN col_name
```

Check for existence to avoid error

```
ALTER TABLE table_name
```

```
DROP COLUMN IF EXISTS col_name
```

Drop multiple columns

```
ALTER TABLE table_name
```

```
DROP COLUMN col_one,
```

```
DROP COLUMN col_two
```

```
ALTER TABLE new_info
```

```
DROP COLUMN people
```

```
ALTER TABLE new_info
```

```
DROP COLUMN IF EXISTS people
```